

Aula: Manipulação do DOM - Tratamento de Eventos em JavaScript

Duração: 150 minutos (2h30min)

1. Eventos em JavaScript (25 minutos)

O que são Eventos?

Eventos são ações ou ocorrências que acontecem no navegador e que podem ser detectadas e manipuladas pelo JavaScript. Eles são fundamentais para criar interatividade em páginas web.

Principais Tipos de Eventos

Eventos de Mouse

- **click**: Disparado quando o usuário clica em um elemento
- **dblclick**: Disparado em um clique duplo
- **mouseenter**: Quando o mouse entra na área do elemento
- **mouseleave**: Quando o mouse sai da área do elemento
- **mousemove**: Quando o mouse se move sobre o elemento

Eventos de Formulário

- **submit**: Disparado quando um formulário é enviado
- **input**: Disparado quando o valor de um input muda
- **change**: Disparado quando o valor muda e o elemento perde o foco
- **focus**: Quando um elemento recebe foco
- **blur**: Quando um elemento perde o foco

Eventos de Teclado

- **keydown**: Quando uma tecla é pressionada
- **keyup**: Quando uma tecla é solta
- **keypress**: Quando uma tecla é pressionada (descontinuado)

Outros Eventos Importantes

- **load**: Quando a página ou recurso termina de carregar
- **scroll**: Quando o usuário rola a página
- **resize**: Quando a janela é redimensionada

Exemplo Básico



html

```
<!DOCTYPE html>
<html>
<head>
  <title>Exemplo de Eventos</title>
</head>
<body>
  <button onclick="alert('Botão clicado!')">Clique aqui</button>
</body>
</html>
```

Observação: Embora o método onclick inline funcione, não é a melhor prática. Veremos formas melhores a seguir.

Exercício 1: Identificando Eventos

Enunciado: Crie uma página HTML com diferentes elementos e teste os eventos mencionados. Adicione um parágrafo que exibe qual evento foi disparado.

Solução:



html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício 1 - Eventos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    .box {
      width: 200px;
      height: 100px;
      background-color: #3498db;
      color: white;
      display: flex;
      align-items: center;
      justify-content: center;
      margin: 10px 0;
      cursor: pointer;
    }
    #resultado {
      margin-top: 20px;
      padding: 10px;
      background-color: #ecf0f1;
      min-height: 50px;
      border-radius: 5px;
    }
    input {
      padding: 8px;
      margin: 10px 0;
      width: 200px;
    }
  </style>
</head>
<body>
  <h1>Testando Eventos</h1>

  <div class="box" onclick="registrarEvento('Click na div')">
    Clique aqui
  </div>

  <div class="box" ondblclick="registrarEvento('Duplo click na div')">
```

```
        Duplo clique aqui
    </div>

    <input type="text"
        placeholder="Digite algo..."
        oninput="registrarEvento('Input detectado: ' + this.value)">

    <input type="text"
        placeholder="Teste o focus"
        onfocus="registrarEvento('Input recebeu foco')"
        onblur="registrarEvento('Input perdeu foco')">

    <div id="resultado">
        <strong>Último evento:</strong> Nenhum ainda
    </div>

    <script>
        function registrarEvento(mensagem) {
            const resultado = document.getElementById('resultado');
            const dataHora = new Date().toLocaleTimeString();
            resultado.innerHTML = `<strong>Último evento (${dataHora}):</strong> ${mensagem}`;
        }
    </script>
</body>
</html>
```

2. addEventListener e Funções de Resposta (30 minutos)

Por que usar addEventListener?

O método `addEventListener` é a forma moderna e recomendada de trabalhar com eventos em JavaScript por diversos motivos:

1. **Separação de responsabilidades:** Mantém HTML e JavaScript separados
2. **Múltiplos listeners:** Permite adicionar vários ouvintes ao mesmo evento
3. **Remoção de eventos:** Possibilita remover eventos quando necessário
4. **Melhor controle:** Oferece opções avançadas de configuração

Sintaxe



javascript

`elemento.addEventListener(tipo, função, opções);`

- **tipo:** String com o nome do evento ('click', 'input', etc.)
- **função:** Função a ser executada quando o evento ocorrer
- **opções:** Parâmetro opcional com configurações

Formas de Definir Funções de Resposta

1. Função Nomeada



javascript

```
function handleClick() {  
    alert('Botão clicado!');  
}  
  
const botao = document.getElementById('meuBotao');  
botao.addEventListener('click', handleClick);
```

2. Função Anônima



javascript

```
const botao = document.getElementById('meuBotao');  
botao.addEventListener('click', function() {  
    alert('Botão clicado!');  
});
```

3. Arrow Function



javascript

```
const botao = document.getElementById('meuBotao');  
botao.addEventListener('click', () => {  
    alert('Botão clicado!');  
});
```

O Objeto Event

Quando um evento é disparado, o JavaScript passa automaticamente um objeto event para a função de resposta:



javascript

```
botao.addEventListener('click', function(event) {  
  console.log('Tipo do evento:', event.type);  
  console.log('Elemento alvo:', event.target);  
  console.log('Coordenadas X:', event.clientX);  
  console.log('Coordenadas Y:', event.clientY);  
});
```

Propriedades Úteis do Event

- **event.target:** O elemento que disparou o evento
- **event.type:** O tipo de evento
- **event.preventDefault():** Previne o comportamento padrão
- **event.stopPropagation():** Para a propagação do evento

Removendo Event Listeners



javascript

```
function handleClick() {  
  console.log('Clicou!');  
}  
  
// Adicionar  
botao.addEventListener('click', handleClick);  
  
// Remover  
botao.removeEventListener('click', handleClick);
```

Importante: Para remover um listener, você precisa passar a mesma referência da função.

Exercício 2: Trabalhando com addEventListener

Enunciado: Crie um contador que aumenta quando um botão é clicado e outro botão que remove o evento do primeiro botão. Adicione também um botão para restaurar o evento.

Solução:



html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício 2 - addEventListener</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
      text-align: center;
    }
    .contador {
      font-size: 48px;
      margin: 20px 0;
      color: #2c3e50;
    }
    button {
      padding: 12px 24px;
      margin: 5px;
      font-size: 16px;
      cursor: pointer;
      border: none;
      border-radius: 5px;
      transition: all 0.3s;
    }
    .btn-incrementar {
      background-color: #27ae60;
      color: white;
    }
    .btn-incrementar:hover {
      background-color: #229954;
    }
    .btn-incrementar:disabled {
      background-color: #95a5a6;
      cursor: not-allowed;
    }
    .btn-remover {
      background-color: #e74c3c;
      color: white;
    }
    .btn-remover:hover {
      background-color: #c0392b;
    }
  </style>
</html>
```

```
.btn-restaurar {
  background-color: #3498db;
  color: white;
}
.btn-restaurar:hover {
  background-color: #2980b9;
}
.btn-reset {
  background-color: #95a5a6;
  color: white;
}
.btn-reset:hover {
  background-color: #7f8c8d;
}
.status {
  margin-top: 20px;
  padding: 10px;
  border-radius: 5px;
  display: inline-block;
}
.ativo {
  background-color: #d5f4e6;
  color: #27ae60;
}
.inativo {
  background-color: #fadbd8;
  color: #e74c3c;
}
</style>
</head>
<body>
  <h1>Contador Interativo</h1>

  <div class="contador" id="contador">0</div>

  <div>
    <button id="btnIncrementar" class="btn-incrementar">
      Incrementar Contador
    </button>
    <button id="btnRemover" class="btn-remover">
      Desativar Incremento
    </button>
    <button id="btnRestaurar" class="btn-restaurar">
      Reativar Incremento
    </button>
  </div>
</body>
</html>
```

```
</button>
<button id="btnReset" class="btn-reset">
  Resetar Contador
</button>
</div>
```

```
<div class="status ativo" id="status">
  Status: <strong>Evento Ativo</strong>
</div>
```

```
<script>
  // Variáveis globais
  let contador = 0;
  let eventoAtivo = true;

  // Elementos
  const contadorEl = document.getElementById('contador');
  const btnIncrementar = document.getElementById('btnIncrementar');
  const btnRemover = document.getElementById('btnRemover');
  const btnRestaurar = document.getElementById('btnRestaurar');
  const btnReset = document.getElementById('btnReset');
  const statusEl = document.getElementById('status');

  // Função que incrementa o contador
  function incrementarContador(event) {
    contador++;
    contadorEl.textContent = contador;

    // Log do evento
    console.log('Contador incrementado!', {
      contador: contador,
      tipo: event.type,
      elemento: event.target
    });
  }

  // Função para atualizar o status visual
  function atualizarStatus(ativo) {
    if (ativo) {
      statusEl.className = 'status ativo';
      statusEl.innerHTML = 'Status: <strong>Evento Ativo</strong>';
      btnIncrementar.disabled = false;
    } else {
      statusEl.className = 'status inativo';
    }
  }
</script>
```

```
statusEl.innerHTML = 'Status: <strong>Evento Inativo</strong>';  
btnIncrementar.disabled = true;  
}  
}
```

// Adicionar o evento inicialmente

```
btnIncrementar.addEventListener('click', incrementarContador);
```

// Remover o evento

```
btnRemover.addEventListener('click', () => {  
  if (eventoAtivo) {  
    btnIncrementar.removeEventListener('click', incrementarContador);  
    eventoAtivo = false;  
    atualizarStatus(false);  
    console.log('Evento removido!');  
  }  
});
```

// Restaurar o evento

```
btnRestaurar.addEventListener('click', () => {  
  if (!eventoAtivo) {  
    btnIncrementar.addEventListener('click', incrementarContador);  
    eventoAtivo = true;  
    atualizarStatus(true);  
    console.log('Evento restaurado!');  
  }  
});
```

// Resetar o contador

```
btnReset.addEventListener('click', () => {  
  contador = 0;  
  contadorEl.textContent = contador;  
  console.log('Contador resetado!');  
});
```

```
</script>
```

```
</body>
```

```
</html>
```

3. Manipulação de Formulários e Atualização de Elementos (30 minutos)

Trabalhando com Formulários

Formulários são uma das principais formas de interação do usuário com a página. O JavaScript permite capturar e manipular dados de formulários de forma dinâmica.

Prevenindo o Comportamento Padrão

O evento submit de um formulário, por padrão, recarrega a página. Para evitar isso:



javascript

```
formulario.addEventListener('submit', function(event) {
    event.preventDefault(); // Previne o recarregamento
    // Seu código aqui
});
```

Acessando Valores de Inputs



javascript

```
// Por ID
const valor = document.getElementById('meuInput').value;

// Por name (em formulários)
const form = document.getElementById('meuForm');
const valor = form.elements['nomeDoInput'].value;

// Usando querySelector
const valor = document.querySelector('#meuInput').value;
```

Tipos de Inputs e Como Acessá-los

Input Text/Email/Number



javascript

```
const texto = document.getElementById('nome').value;
```

Checkbox



javascript

```
const checkbox = document.getElementById('aceito');
const estaMarcado = checkbox.checked; // true ou false
```

Radio Buttons



javascript

```
const radios = document.getElementsByName('genero');
let selecionado;
radios.forEach(radio => {
  if (radio.checked) {
    selecionado = radio.value;
  }
});
```

Select (Dropdown)



javascript

```
const select = document.getElementById('estado');
const valorSelecioneado = select.value;
const textoSelecioneado = select.options[select.selectedIndex].text;
```

Atualizando Elementos Dinamicamente

Alterando Texto



javascript

```
elemento.textContent = 'Novo texto'; // Texto puro
elemento.innerHTML = '<strong>Texto com HTML</strong>'; // Com HTML
```

Alterando Atributos



javascript

```
elemento.setAttribute('class', 'nova-classe');
elemento.src = 'nova-imagem.jpg';
elemento.disabled = true;
```

Alterando Estilos



javascript

```
elemento.style.color = 'red';
elemento.style.backgroundColor = 'yellow';
elemento.style.display = 'none';
```

Adicionando/Removendo Classes



javascript

```
elemento.classList.add('classe-nova');
elemento.classList.remove('classe-antiga');
elemento.classList.toggle('classe-alternada');
elemento.classList.contains('classe-verificar'); // retorna true/false
```

 Exercício 3: Formulário Dinâmico

Enunciado: Crie um formulário de cadastro que, ao ser enviado, exibe os dados em uma área abaixo sem recarregar a página. O formulário deve ter: nome, email, idade e uma lista de interesses (checkboxes).

Solução:



html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício 3 - Formulário Dinâmico</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      padding: 20px;
    }
    .container {
      max-width: 800px;
      margin: 0 auto;
    }
    h1 {
      color: white;
      text-align: center;
      margin-bottom: 30px;
      text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
    }
    .form-container {
      background: white;
      padding: 30px;
      border-radius: 15px;
      box-shadow: 0 10px 40px rgba(0,0,0,0.2);
      margin-bottom: 30px;
    }
    .form-group {
      margin-bottom: 20px;
    }
    label {
      display: block;
      margin-bottom: 8px;
      color: #333;
      font-weight: 600;
    }
```

```
input[type="text"],
input[type="email"],
input[type="number"] {
  width: 100%;
  padding: 12px;
  border: 2px solid #e0e0e0;
  border-radius: 8px;
  font-size: 16px;
  transition: border-color 0.3s;
}
input[type="text"]:focus,
input[type="email"]:focus,
input[type="number"]:focus {
  outline: none;
  border-color: #667eea;
}
.checkbox-group {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
}
.checkbox-item {
  display: flex;
  align-items: center;
  gap: 8px;
}
input[type="checkbox"] {
  width: 18px;
  height: 18px;
  cursor: pointer;
}
.checkbox-item label {
  margin: 0;
  cursor: pointer;
  font-weight: normal;
}
button {
  width: 100%;
  padding: 14px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 18px;
```

```
font-weight: 600;
cursor: pointer;
transition: transform 0.2s, box-shadow 0.2s;
}
button:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 20px rgba(102, 126, 234, 0.4);
}
button:active {
  transform: translateY(0);
}
#resultado {
  background: white;
  padding: 30px;
  border-radius: 15px;
  box-shadow: 0 10px 40px rgba(0,0,0,0.2);
  display: none;
}
#resultado.visivel {
  display: block;
  animation: slideIn 0.5s ease-out;
}
@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
#resultado h2 {
  color: #667eea;
  margin-bottom: 20px;
  border-bottom: 3px solid #667eea;
  padding-bottom: 10px;
}
.info-item {
  margin-bottom: 15px;
  padding: 12px;
  background: #f8f9fa;
  border-radius: 8px;
  border-left: 4px solid #667eea;
```

```
}
.info-item strong {
  color: #764ba2;
  display: inline-block;
  min-width: 100px;
}
.interesses-lista {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 8px;
}
.interesse-tag {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 6px 12px;
  border-radius: 20px;
  font-size: 14px;
}
</style>
</head>
<body>
<div class="container">
  <h1> 📋 Formulário de Cadastro</h1>

  <div class="form-container">
    <form id="formCadastro">
      <div class="form-group">
        <label for="nome">Nome Completo:</label>
        <input type="text" id="nome" name="nome" required>
      </div>

      <div class="form-group">
        <label for="email">E-mail:</label>
        <input type="email" id="email" name="email" required>
      </div>

      <div class="form-group">
        <label for="idade">Idade:</label>
        <input type="number" id="idade" name="idade" min="1" max="120" required>
      </div>

      <div class="form-group">
        <label>Interesses:</label>
```

```
<div class="checkbox-group">
  <div class="checkbox-item">
    <input type="checkbox" id="programacao" name="interesses" value="Programação">
    <label for="programacao">Programação</label>
  </div>
  <div class="checkbox-item">
    <input type="checkbox" id="design" name="interesses" value="Design">
    <label for="design">Design</label>
  </div>
  <div class="checkbox-item">
    <input type="checkbox" id="musica" name="interesses" value="Música">
    <label for="musica">Música</label>
  </div>
  <div class="checkbox-item">
    <input type="checkbox" id="esportes" name="interesses" value="Esportes">
    <label for="esportes">Esportes</label>
  </div>
  <div class="checkbox-item">
    <input type="checkbox" id="leitura" name="interesses" value="Leitura">
    <label for="leitura">Leitura</label>
  </div>
  <div class="checkbox-item">
    <input type="checkbox" id="viagens" name="interesses" value="Viagens">
    <label for="viagens">Viagens</label>
  </div>
</div>

<button type="submit">Cadastrar</button>
</form>
</div>

<div id="resultado">
  <h2>✔ Cadastro Realizado com Sucesso!</h2>
  <div id="dadosExibidos"></div>
</div>
</div>

<script>
  // Capturar o formulário
  const form = document.getElementById('formCadastro');
  const resultado = document.getElementById('resultado');
  const dadosExibidos = document.getElementById('dadosExibidos');
```

// Adicionar evento de submit

```
form.addEventListener('submit', function(event) {  
    // Prevenir o comportamento padrão (recarregar a página)  
    event.preventDefault();
```

// Capturar os valores dos campos

```
const nome = document.getElementById('nome').value;  
const email = document.getElementById('email').value;  
const idade = document.getElementById('idade').value;
```

// Capturar interesses selecionados

```
const checkboxes = document.querySelectorAll('input[name="interesses"]:checked');  
const interesses = [];  
checkboxes.forEach(checkbox => {  
    interesses.push(checkbox.value);  
});
```

// Montar o HTML com os dados

```
let html = `  
    <div class="info-item">  
        <strong>Nome:</strong> ${nome}  
    </div>  
    <div class="info-item">  
        <strong>E-mail:</strong> ${email}  
    </div>  
    <div class="info-item">  
        <strong>Idade:</strong> ${idade} anos  
    </div>  
    <div class="info-item">  
        <strong>Interesses:</strong>  
`;  
;
```

```
if (interesses.length > 0) {  
    html += '<div class="interesses-lista">';  
    interesses.forEach(interesse => {  
        html += '<span class="interesse-tag">${interesse}</span>';  
    });  
    html += '</div>';  
} else {  
    html += '<p>Nenhum interesse selecionado</p>';  
}
```

```
html += '</div>';
```

```
// Exibir os dados
dadosExibidos.innerHTML = html;
resultado.classList.add('visivel');

// Rolar suavemente até o resultado
resultado.scrollIntoView({ behavior: 'smooth', block: 'nearest' });

// Limpar o formulário
form.reset();

// Log no console
console.log('Cadastro realizado:', {
  nome,
  email,
  idade,
  interesses
});
});
</script>
</body>
</html>
```

4. Validação Simples de Campos usando Eventos (30 minutos)

Importância da Validação

A validação de formulários é essencial para:

- Garantir que os dados estejam no formato correto
- Melhorar a experiência do usuário com feedback imediato
- Prevenir erros antes do envio
- Reduzir requisições inválidas ao servidor

Eventos Úteis para Validação

- **input**: Valida enquanto o usuário digita
- **blur**: Valida quando o campo perde o foco
- **submit**: Validação final antes do envio

Técnicas de Validação

1. Validação de Campo Vazio



javascript

```
if (input.value.trim() === '') {  
  mostrarErro('Campo obrigatório');  
}
```

2. Validação de Tamanho



javascript

```
if (input.value.length < 3) {  
  mostrarErro('Mínimo de 3 caracteres');  
}
```

3. Validação com Expressões Regulares



javascript

```
const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
if (!regexEmail.test(email.value)) {  
  mostrarErro('E-mail inválido');  
}
```

4. Validação de Números



javascript

```
const idade = parseInt(input.value);  
if (isNaN(idade) || idade < 18 || idade > 100) {  
  mostrarErro('Idade deve estar entre 18 e 100');  
}
```

Feedback Visual

É importante dar feedback visual ao usuário:



javascript

```
function mostrarErro(input, mensagem) {
  input.classList.add('erro');
  const msgErro = document.createElement('span');
  msgErro.classList.add('mensagem-erro');
  msgErro.textContent = mensagem;
  input.parentElement.appendChild(msgErro);
}

function limparErro(input) {
  input.classList.remove('erro');
  const msgErro = input.parentElement.querySelector('.mensagem-erro');
  if (msgErro) {
    msgErro.remove();
  }
}
```

Validação em Tempo Real



javascript

```
input.addEventListener('input', function() {
  limparErro(this);
  if (this.value.length < 3) {
    mostrarErro(this, 'Mínimo de 3 caracteres');
  }
});
```

Exercício 4: Formulário com Validação

Enunciado: Crie um formulário de registro com validação completa em tempo real para: nome (mínimo 3 caracteres), email (formato válido), senha (mínimo 8 caracteres, deve conter letra e número) e confirmação de senha (deve ser igual à senha).

Solução:



html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício 4 - Validação de Formulário</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #1e3c72 0%, #2a5298 100%);
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }
    .container {
      background: white;
      padding: 40px;
      border-radius: 20px;
      box-shadow: 0 15px 50px rgba(0,0,0,0.3);
      max-width: 500px;
      width: 100%;
    }
    h1 {
      color: #1e3c72;
      margin-bottom: 10px;
      text-align: center;
    }
    .subtitle {
      text-align: center;
      color: #666;
      margin-bottom: 30px;
      font-size: 14px;
    }
    .form-group {
      margin-bottom: 25px;
      position: relative;
    }
```

```
label {
  display: block;
  margin-bottom: 8px;
  color: #333;
  font-weight: 600;
  font-size: 14px;
}
input {
  width: 100%;
  padding: 12px 15px;
  border: 2px solid #e0e0e0;
  border-radius: 8px;
  font-size: 16px;
  transition: all 0.3s;
}
input:focus {
  outline: none;
  border-color: #2a5298;
}
input.valido {
  border-color: #27ae60;
  background-color: #f0fff4;
}
input.invalido {
  border-color: #e74c3c;
  background-color: #fff5f5;
}
.mensagem-erro {
  display: block;
  color: #e74c3c;
  font-size: 12px;
  margin-top: 5px;
  font-weight: 500;
}
.mensagem-sucesso {
  display: block;
  color: #27ae60;
  font-size: 12px;
  margin-top: 5px;
  font-weight: 500;
}
.icone-validacao {
  position: absolute;
  right: 15px;
```

```
    top: 43px;
    font-size: 20px;
}
button {
    width: 100%;
    padding: 15px;
    background: linear-gradient(135deg, #1e3c72 0%, #2a5298 100%);
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 18px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s;
    margin-top: 10px;
}
button:hover:not(:disabled) {
    transform: translateY(-2px);
    box-shadow: 0 5px 20px rgba(30, 60, 114, 0.4);
}
button:disabled {
    background: #95a5a6;
    cursor: not-allowed;
}
.requisitos {
    background: #f8f9fa;
    padding: 15px;
    border-radius: 8px;
    margin-top: 5px;
    font-size: 12px;
}
.requisitos ul {
    margin: 8px 0 0 20px;
}
.requisitos li {
    margin: 5px 0;
    color: #666;
}
.requisitos li.cumprido {
    color: #27ae60;
    font-weight: 600;
}
.sucesso-final {
    display: none;
```

```

background: #d4edda;
border: 2px solid #27ae60;
padding: 20px;
border-radius: 8px;
text-align: center;
margin-top: 20px;
}
.sucesso-final.visivel {
display: block;
animation: slideIn 0.5s ease-out;
}
@keyframes slideIn {
from {
opacity: 0;
transform: scale(0.9);
}
to {
opacity: 1;
transform: scale(1);
}
}
.sucesso-final h2 {
color: #27ae60;
margin-bottom: 10px;
}
</style>
</head>
<body>
<div class="container">
<h1>🔒 Criar Conta</h1>
<p class="subtitle">Preencha todos os campos corretamente</p>

<form id="formRegistro">
<div class="form-group">
<label for="nome">Nome Completo:</label>
<input type="text" id="nome" name="nome" placeholder="Digite seu nome completo">
<span class="icone-validacao" id="iconeNome"></span>
<span class="mensagem-erro" id="erroNome"></span>
<span class="mensagem-sucesso" id="sucessoNome"></span>
</div>

<div class="form-group">
<label for="email">E-mail:</label>
<input type="email" id="email" name="email" placeholder="seu@email.com">

```

```
<span class="icone-validacao" id="iconeEmail"></span>
<span class="mensagem-erro" id="erroEmail"></span>
<span class="mensagem-sucesso" id="sucessoEmail"></span>
</div>
```

```
<div class="form-group">
  <label for="senha">Senha:</label>
  <input type="password" id="senha" name="senha" placeholder="Digite uma senha forte">
  <span class="icone-validacao" id="iconeSenha"></span>
  <span class="mensagem-erro" id="erroSenha"></span>
  <div class="requisitos">
    <strong>Requisitos da senha:</strong>
    <ul>
      <li id="req-tamanho">Mínimo de 8 caracteres</li>
      <li id="req-letra">Pelo menos uma letra</li>
      <li id="req-numero">Pelo menos um número</li>
    </ul>
  </div>
</div>
```

```
<div class="form-group">
  <label for="confirmarSenha">Confirmar Senha:</label>
  <input type="password" id="confirmarSenha" name="confirmarSenha" placeholder="Digite a senha novamente">
  <span class="icone-validacao" id="iconeConfirmar"></span>
  <span class="mensagem-erro" id="erroConfirmar"></span>
  <span class="mensagem-sucesso" id="sucessoConfirmar"></span>
</div>
```

```
<button type="submit" id="btnSubmit" disabled>Criar Conta</button>
</form>
```

```
<div class="sucesso-final" id="sucessoFinal">
  <h2>✔ Conta criada com sucesso!</h2>
  <p>Bem-vindo(a) à plataforma!</p>
</div>
</div>
```

```
<script>
  // Elementos do formulário
  const form = document.getElementById('formRegistro');
  const nome = document.getElementById('nome');
  const email = document.getElementById('email');
  const senha = document.getElementById('senha');
  const confirmarSenha = document.getElementById('confirmarSenha');
```

```
const btnSubmit = document.getElementById('btnSubmit');
```

```
// Estado de validação
```

```
const validacao = {  
  nome: false,  
  email: false,  
  senha: false,  
  confirmarSenha: false  
};
```

```
// Função para atualizar o botão de submit
```

```
function atualizarBotao() {  
  const todosValidos = Object.values(validacao).every(v => v === true);  
  btnSubmit.disabled = !todosValidos;  
}
```

```
// Função para mostrar erro
```

```
function mostrarErro(campo, mensagem) {  
  const input = document.getElementById(campo);  
  const erro = document.getElementById(`erro${campo.charAt(0).toUpperCase() + campo.slice(1)}`);  
  const sucesso = document.getElementById(`sucesso${campo.charAt(0).toUpperCase() + campo.slice(1)}`);  
  const icone = document.getElementById(`icone${campo.charAt(0).toUpperCase() + campo.slice(1)}`);  
  
  input.classList.remove('valido');  
  input.classList.add('invalido');  
  erro.textContent = mensagem;  
  if (sucesso) sucesso.textContent = "";  
  if (icone) icone.textContent = '✖';  
  
  validacao[campo] = false;  
  atualizarBotao();  
}
```

```
// Função para mostrar sucesso
```

```
function mostrarSucesso(campo, mensagem = "") {  
  const input = document.getElementById(campo);  
  const erro = document.getElementById(`erro${campo.charAt(0).toUpperCase() + campo.slice(1)}`);  
  const sucesso = document.getElementById(`sucesso${campo.charAt(0).toUpperCase() + campo.slice(1)}`);  
  const icone = document.getElementById(`icone${campo.charAt(0).toUpperCase() + campo.slice(1)}`);  
  
  input.classList.remove('invalido');  
  input.classList.add('valido');  
  erro.textContent = "";  
  if (sucesso && mensagem) sucesso.textContent = mensagem;
```

```

if (icone) icone.textContent = '✓';

validacao[campo] = true;
atualizarBotao();
}

// Função para limpar validação
function limparValidacao(campo) {
    const input = document.getElementById(campo);
    const erro = document.getElementById(`erro${campo.charAt(0).toUpperCase() + campo.slice(1)}`);
    const sucesso = document.getElementById(`sucesso${campo.charAt(0).toUpperCase() + campo.slice(1)}`);
    const icone = document.getElementById(`icone${campo.charAt(0).toUpperCase() + campo.slice(1)}`);

    input.classList.remove('valido', 'invalido');
    erro.textContent = "";
    if (sucesso) sucesso.textContent = "";
    if (icone) icone.textContent = "";
}

// VALIDAÇÃO DO NOME
nome.addEventListener('input', function() {
    const valor = this.value.trim();

    if (valor === "") {
        limparValidacao('nome');
        validacao.nome = false;
        atualizarBotao();
        return;
    }

    if (valor.length < 3) {
        mostrarErro('nome', 'O nome deve ter pelo menos 3 caracteres');
    } else if (!/^[a-zA-ZÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖÜÇÑ\S]+$/.test(valor)) {
        mostrarErro('nome', 'O nome deve conter apenas letras');
    } else {
        mostrarSucesso('nome', '✓ Nome válido');
    }
});

// VALIDAÇÃO DO EMAIL
email.addEventListener('input', function() {
    const valor = this.value.trim();

    if (valor === "") {

```

```
limparValidacao('email');
validacao.email = false;
atualizarBotao();
return;
}

const regexEmail = /^[^s@]+@[^s@]+\.[^s@]+$/;

if (!regexEmail.test(valor)) {
    mostrarErro('email', 'Digite um e-mail válido (ex: usuario@email.com)');
} else {
    mostrarSucesso('email', '✓ E-mail válido');
}
});
```

// VALIDAÇÃO DA SENHA

```
senha.addEventListener('input', function() {
    const valor = this.value;

    // Elementos dos requisitos
    const reqTamanho = document.getElementById('req-tamanho');
    const reqLetra = document.getElementById('req-letra');
    const reqNumero = document.getElementById('req-numero');

    if (valor === "") {
        limparValidacao('senha');
        validacao.senha = false;
        reqTamanho.classList.remove('cumprido');
        reqLetra.classList.remove('cumprido');
        reqNumero.classList.remove('cumprido');
        atualizarBotao();
        return;
    }

    // Verificar requisitos
    const temTamanho = valor.length >= 8;
    const temLetra = /[a-zA-Z]/.test(valor);
    const temNumero = /[0-9]/.test(valor);

    // Atualizar visual dos requisitos
    reqTamanho.classList.toggle('cumprido', temTamanho);
    reqLetra.classList.toggle('cumprido', temLetra);
    reqNumero.classList.toggle('cumprido', temNumero);
```

// Validar

```
if (!itemTamanho) {  
    mostrarErro('senha', 'A senha deve ter pelo menos 8 caracteres');  
} else if (!itemLetra) {  
    mostrarErro('senha', 'A senha deve conter pelo menos uma letra');  
} else if (!itemNumero) {  
    mostrarErro('senha', 'A senha deve conter pelo menos um número');  
} else {  
    mostrarSucesso('senha');
```

// Revalidar confirmação de senha se já foi preenchida

```
if (confirmarSenha.value !== "") {  
    confirmarSenha.dispatchEvent(new Event('input'));  
}  
});
```

// VALIDAÇÃO DA CONFIRMAÇÃO DE SENHA

```
confirmarSenha.addEventListener('input', function() {  
    const valor = this.value;  
  
    if (valor === "") {  
        limparValidacao('confirmarSenha');  
        validacao.confirmarSenha = false;  
        atualizarBotao();  
        return;  
    }  
  
    if (valor !== senha.value) {  
        mostrarErro('confirmarSenha', 'As senhas não coincidem');  
    } else if (senha.value === "") {  
        mostrarErro('confirmarSenha', 'Digite a senha primeiro');  
    } else {  
        mostrarSucesso('confirmarSenha', '✓ Senhas coincidem');  
    }  
});
```

// SUBMIT DO FORMULÁRIO

```
form.addEventListener('submit', function(event) {  
    event.preventDefault();
```

// Verificar se todos os campos são válidos

```
const todosValidos = Object.values(validacao).every(v => v === true);
```

```
if (!todosValidos) {
    alert('Por favor, preencha todos os campos corretamente!');
    return;
}

// Exibir mensagem de sucesso
const sucessoFinal = document.getElementById('sucessoFinal');
sucessoFinal.classList.add('visivel');

// Esconder o formulário
form.style.display = 'none';

// Log dos dados (em produção, enviaria para o servidor)
console.log('Formulário enviado com sucesso!', {
    nome: nome.value,
    email: email.value,
    senha: '[PROTEGIDA]'
});
});
</script>
</body>
</html>
```

5. Eventos de Teclado e Outros Eventos Interessantes (35 minutos)

Eventos de Teclado

Os eventos de teclado permitem capturar e responder às interações do usuário com o teclado.

Tipos de Eventos de Teclado

keydown: Disparado quando uma tecla é pressionada (antes de aparecer o caractere) **keyup:** Disparado quando uma tecla é solta **keypress:** Disparado quando uma tecla de caractere é pressionada (obsoleto)

Propriedades Importantes do Evento de Teclado



javascript

```
elemento.addEventListener('keydown', function(event) {
  console.log('Tecla:', event.key);      // A tecla pressionada ('a', 'Enter', etc.)
  console.log('Código:', event.code);    // Código físico da tecla ('KeyA', 'Enter')
  console.log('Ctrl:', event.ctrlKey);   // true se Ctrl estiver pressionado
  console.log('Shift:', event.shiftKey); // true se Shift estiver pressionado
  console.log('Alt:', event.altKey);     // true se Alt estiver pressionado
  console.log('Key Code:', event.keyCode); // Código numérico (obsoleto)
});
```

Teclas Especiais

- **Enter:** event.key === 'Enter'
- **Escape:** event.key === 'Escape'
- **Setas:** 'ArrowUp', 'ArrowDown', 'ArrowLeft', 'ArrowRight'
- **Espaço:** event.key === ' ' ou event.code === 'Space'
- **Tab:** event.key === 'Tab'

Combinações de Teclas



javascript

```
document.addEventListener('keydown', function(event) {
  // Ctrl + S
  if (event.ctrlKey && event.key === 's') {
    event.preventDefault(); // Previne salvar página
    salvarDados();
  }

  // Ctrl + Shift + D
  if (event.ctrlKey && event.shiftKey && event.key === 'D') {
    debugMode();
  }
});
```

Outros Eventos Interessantes

1. Eventos de Scroll



javascript

```
window.addEventListener('scroll', function() {  
    const scrollY = window.scrollY;  
    const scrollPercentage = (scrollY / (document.body.scrollHeight - window.innerHeight)) * 100;  
    console.log('Scroll:', scrollPercentage.toFixed(2) + '%');  
});
```

2. Eventos de Resize



javascript

```
window.addEventListener('resize', function() {  
    console.log('Largura:', window.innerWidth);  
    console.log('Altura:', window.innerHeight);  
});
```

3. Evento de Copy/Paste



javascript

```
elemento.addEventListener('copy', function(event) {  
    event.preventDefault();  
    console.log("Tentou copiar!");  
});
```

```
elemento.addEventListener('paste', function(event) {  
    event.preventDefault();  
    const texto = event.clipboardData.getData('text');  
    console.log('Colou:', texto);  
});
```

4. Evento de ContextMenu (Botão Direito)



javascript

```
elemento.addEventListener('contextmenu', function(event) {  
    event.preventDefault();  
    console.log('Clicou com botão direito!');  
    // Pode exibir menu personalizado  
});
```

5. Eventos de Drag and Drop



javascript

```
elemento.addEventListener('dragstart', function(event) {  
    event.dataTransfer.setData('text', event.target.id);  
});  
  
alvo.addEventListener('drop', function(event) {  
    event.preventDefault();  
    const data = event.dataTransfer.getData('text');  
    console.log('Soltou:', data);  
});  
  
alvo.addEventListener('dragover', function(event) {  
    event.preventDefault(); // Necessário para permitir o drop  
});
```

6. Evento de Visibility Change



javascript

```
document.addEventListener('visibilitychange', function() {  
    if (document.hidden) {  
        console.log('Usuário saiu da aba');  
    } else {  
        console.log('Usuário voltou para a aba');  
    }  
});
```

7. Eventos de Mouse Adicionais



javascript

```
elemento.addEventListener('mouseenter', function() {  
    console.log('Mouse entrou');  
});  
  
elemento.addEventListener('mouseleave', function() {  
    console.log('Mouse saiu');  
});  
  
elemento.addEventListener('mousemove', function(event) {  
    console.log('X:', event.clientX, 'Y:', event.clientY);  
});
```



Exercício 5: Aplicação com Eventos Diversos

Enunciado: Crie uma aplicação interativa que demonstre diversos tipos de eventos: um campo de busca que responde ao teclado, uma área arrastável, um contador de scrolls, e atalhos de teclado.

Solução:



html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício 5 - Eventos Diversos</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 200vh;
      padding: 20px;
    }
    .container {
      max-width: 1200px;
      margin: 0 auto;
    }
    h1 {
      color: white;
      text-align: center;
      margin-bottom: 30px;
      text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
    }
    .grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
      gap: 20px;
      margin-bottom: 30px;
    }
    .card {
      background: white;
      padding: 25px;
      border-radius: 15px;
      box-shadow: 0 10px 30px rgba(0,0,0,0.2);
    }
    .card h2 {
      color: #667eea;
      margin-bottom: 15px;
      font-size: 20px;
```

```
}
```

```
/* Busca com teclado */
```

```
#campoBusca {  
    width: 100%;  
    padding: 12px;  
    border: 2px solid #e0e0e0;  
    border-radius: 8px;  
    font-size: 16px;  
    margin-bottom: 10px;  
}
```

```
#campoBusca:focus {  
    outline: none;  
    border-color: #667eea;  
}
```

```
.info-tecla {  
    background: #f0f0f0;  
    padding: 10px;  
    border-radius: 5px;  
    font-size: 14px;  
    margin-top: 10px;  
}
```

```
.info-tecla span {  
    font-weight: bold;  
    color: #667eea;  
}
```

```
/* Área arrastável */
```

```
.drag-container {  
    min-height: 200px;  
    background: #f8f9fa;  
    border: 3px dashed #ccc;  
    border-radius: 10px;  
    padding: 20px;  
    position: relative;  
}
```

```
.drag-item {  
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
    color: white;  
    padding: 15px 20px;  
    border-radius: 10px;  
    cursor: move;  
    display: inline-block;  
    margin: 5px;
```

```
    font-weight: 600;
    box-shadow: 0 4px 10px rgba(0,0,0,0.2);
    transition: transform 0.2s;
}
.drag-item:hover {
    transform: scale(1.05);
}
.drag-item.dragging {
    opacity: 0.5;
}
.drop-zone {
    min-height: 150px;
    background: #e8f5e9;
    border: 3px dashed #4caf50;
    border-radius: 10px;
    padding: 20px;
    margin-top: 15px;
    text-align: center;
    color: #4caf50;
    font-weight: 600;
}
.drop-zone.over {
    background: #c8e6c9;
}

/* Contador de scroll */
.scroll-info {
    background: #f8f9fa;
    padding: 15px;
    border-radius: 8px;
    text-align: center;
}
.scroll-bar {
    width: 100%;
    height: 20px;
    background: #e0e0e0;
    border-radius: 10px;
    overflow: hidden;
    margin-top: 10px;
}
.scroll-progress {
    height: 100%;
    background: linear-gradient(90deg, #667eea 0%, #764ba2 100%);
    width: 0%;
```

```
    transition: width 0.1s;
}

/* Atalhos */
.atalhos-lista {
    list-style: none;
}
.atalhos-lista li {
    padding: 10px;
    margin: 5px 0;
    background: #f8f9fa;
    border-radius: 5px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.tecla {
    background: #667eea;
    color: white;
    padding: 5px 10px;
    border-radius: 5px;
    font-size: 12px;
    font-weight: bold;
}

/* Log de eventos */
.log {
    background: #2c3e50;
    color: #ecf0f1;
    padding: 15px;
    border-radius: 8px;
    max-height: 200px;
    overflow-y: auto;
    font-family: 'Courier New', monospace;
    font-size: 12px;
}
.log-item {
    padding: 5px;
    border-bottom: 1px solid #34495e;
}
.log-item:last-child {
    border-bottom: none;
}
.log-time {
```

```
        color: #3498db;
    }

    /* Posição fixa para scroll */
    .scroll-indicator {
        position: fixed;
        top: 0;
        left: 0;
        right: 0;
        height: 5px;
        background: #e0e0e0;
        z-index: 1000;
    }
    .scroll-indicator-bar {
        height: 100%;
        background: linear-gradient(90deg, #667eea 0%, #764ba2 100%);
        width: 0%;
        transition: width 0.1s;
    }
</style>
</head>
<body>
    <div class="scroll-indicator">
        <div class="scroll-indicator-bar" id="scrollIndicatorBar"></div>
    </div>

    <div class="container">
        <h1>📖 Demonstração de Eventos Diversos</h1>

        <div class="grid">
            <!-- Card 1: Eventos de Teclado -->
            <div class="card">
                <h2>📖 Eventos de Teclado</h2>
                <input type="text" id="campoBusca" placeholder="Digite algo... (ESC para limpar)">
                <div class="info-tecla" id="infoTecla">
                    Aguardando digitação...
                </div>
                <div class="info-tecla" style="margin-top: 10px;">
                    <strong>Atalhos:</strong><br>
                    • ESC: Limpar campo<br>
                    • Ctrl+S: Salvar (simulado)<br>
                    • Ctrl+K: Focar busca
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```

<!-- Card 2: Drag and Drop -->

`<div class="card">`

`<h2>🎯 Drag and Drop</h2>`

`<div class="drag-container">`

`<div class="drag-item" draggable="true" data-id="1">📦 Item 1</div>`

`<div class="drag-item" draggable="true" data-id="2">📦 Item 2</div>`

`<div class="drag-item" draggable="true" data-id="3">📦 Item 3</div>`

`</div>`

`<div class="drop-zone" id="dropZone">`

Arraste os itens aqui

`</div>`

`</div>`

<!-- Card 3: Scroll -->

`<div class="card">`

`<h2>📄 Informações de Scroll</h2>`

`<div class="scroll-info">`

`<div>Posição Y: <strong id="scrollY">0</strong>px</div>`

`<div>Porcentagem: <strong id="scrollPercent">0</strong>%</div>`

`<div class="scroll-bar">`

`<div class="scroll-progress" id="scrollProgress"></div>`

`</div>`

`</div>`

`<p style="margin-top: 15px; font-size: 14px; color: #666;">`

Role a página para ver as informações atualizarem em tempo real!

`</p>`

`</div>`

`</div>`

<!-- Log de Eventos -->

`<div class="card">`

`<h2>📋 Log de Eventos (últimos 10)</h2>`

`<div class="log" id="logEventos">`

`<div class="log-item">Aguardando eventos...</div>`

`</div>`

`</div>`

`</div>`

`<script>`

// ===== SISTEMA DE LOG =====

const logEventos = document.getElementById('logEventos');

const logs = [];

```

function adicionarLog(mensagem) {
  const agora = new Date();
  const hora = agora.toLocaleTimeString();

  logs.unshift(`[${hora}] ${mensagem}`);
  if (logs.length > 10) logs.pop();

  logEventos.innerHTML = logs
    .map(log => {
      const [time, ...rest] = log.split(' ');
      return `<div class="log-item"><span class="log-time">${time}</span> ${rest.join(' ')}</div>`;
    })
    .join("");
}

// ===== EVENTOS DE TECLADO =====
const campoBusca = document.getElementById('campoBusca');
const infoTecla = document.getElementById('infoTecla');

campoBusca.addEventListener('keydown', function(event) {
  let info = `Tecla: <span>${event.key}</span> | `;
  info += `Código: <span>${event.code}</span>`;

  if (event.ctrlKey) info += ` | <span>Ctrl pressionado</span>`;
  if (event.shiftKey) info += ` | <span>Shift pressionado</span>`;
  if (event.altKey) info += ` | <span>Alt pressionado</span>`;

  infoTecla.innerHTML = info;
  adicionarLog(`Teclado: ${event.key}`);
});

campoBusca.addEventListener('input', function() {
  adicionarLog(`Busca: "${this.value}"`);
});

// Atalhos de teclado globais
document.addEventListener('keydown', function(event) {
  // ESC para limpar
  if (event.key === 'Escape') {
    campoBusca.value = "";
    campoBusca.blur();
    adicionarLog('Atalho: ESC - Campo limpo');
  }
}

```

```
// Ctrl+S para salvar
if (event.ctrlKey && event.key === 's') {
  event.preventDefault();
  adicionarLog('Atalho: Ctrl+S - Salvamento simulado');
  alert('Dados salvos! (simulação)');
}
```

```
// Ctrl+K para focar busca
if (event.ctrlKey && event.key === 'k') {
  event.preventDefault();
  campoBusca.focus();
  adicionarLog('Atalho: Ctrl+K -
```